



UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA

Medfinder: Web de directorio médico

INF331 - Pruebas de Software

Fecha:	20 de mayo de 2023
Autores:	Cristian Bruna, 201630019-8 Roberto Covarrubias, 201804181-5 Carlos Jara, 201773038-2 Esteban Manríquez, 201673020-6
Profesor:	Oscar Reyes
Paralelo:	1
Link a GitHub:	Medfinder
Link a video:	Presentación Medfinder

Índice

Índice de figuras	1
1 Introducción	1
2 Supuestos	2
3 Alcances de la herramienta	3
4 Trabajo realizado	4
4.1 Proyecto	4
4.2 Dependencias entre herramienta y aplicación	4
5 Pruebas	5
5.1 Estrategia de pruebas utilizadas	5
5.2 Definición de Pruebas	5
5.2.1 Pruebas unitarias/integración	5
5.2.2 Pruebas de Valores límite/Supuestos	5
5.3 Procedimiento de ejecución de pruebas	5
5.3.1 Pruebas unitarias del sistema CRUD	5
5.3.2 Pruebas unitarias de Valores Límite/ Supuestos	6
5.4 Reporte de resultados	7
6 Problemas encontrados y soluciones	9
7 Conclusión	10

1. Introducción

El centro médico "Salud y Bienestar" ofrece diversos servicios de salud con un equipo de profesionales médicos y enfermeras en diferentes áreas como medicina general y pediatría.

Muchos pacientes del centro médico tienen dificultades para encontrar especialistas médicos adecuados y desconocen la ubicación de los especialistas cercanos. Además, algunos pacientes desean conocer los antecedentes y credenciales de los médicos antes de hacer una cita.

Para resolver este problema, el centro médico en este proyecto desarrollará una web de directorio médico que incluirá una lista de todos los especialistas disponibles en la clínica, sus áreas de especialización y permitirá a los pacientes buscar especialistas por especialidad, nombre y ubicación geográfica, además de dar acceso a la visualización de las credenciales del médico.

Dada la relevancia de la estética se utilizará React como tecnología principal de desarrollo Front-End, Django Rest Framework para montar el Back-End manejando SQLite como la base de datos del proyecto. En secciones posteriores, se definirán los supuestos, consideraciones y pruebas realizadas para asegurar la calidad del software desarrollado.

2. Supuestos

- Al llenar el campo *credencial*, se asume que se colocará un link a un documento PDF existente.
- Al llenar el campo *ubicacion*, debe hacerse mediante un link de google maps (Específicamente el link que se obtiene mediante la funcionalidad compartir ubicación de google maps).
- A la hora de ingresar una *especialidad*, se asume que se escribirá alguna de las existentes dentro del sistema.
- Solamente un administrador logeado dentro del sistema podrá agregar, cambiar o eliminar información de los médicos de la plataforma.

3. Alcances de la herramienta

Robot Framework es un marco de automatización de pruebas de código abierto y genérico. Proporciona una sintaxis fácil de usar y legible para la creación de pruebas, lo que lo convierte en una gran herramienta de testeo de Software, que actúa sobre cualquier tipo de app web sin importar su procedencia y lenguaje de programación ocupado para desarrollarla. Además Robot Framework no requiere de conocimientos previos de programación para ser utilizada, debido a su estructura de comandos fáciles y legibles para el tester.

A partir de la naturaleza de la herramienta Robot Framework, las pruebas fueron desarrolladas para ser testeadas en el Front-End de la aplicación.

Por consiguiente, para verificar/comprobar validaciones del Back-End, éstas mismas fueron representadas en el Front-End en formato de alerts que ayudan a tener una mejor cobertura del testeo del Software.

4. Trabajo realizado

4.1. Proyecto

En primer lugar escogimos el stack de tecnologías a utilizar, tanto en el desarrollo del Software como a la hora de probarlo. De aquí escogimos Django Rest Framework para montar el Back-End, React para desarrollar el Front-End y SQLite como Base de Datos.

En un principio la herramienta escogida para testear la aplicación fué Enzyme, que es una herramienta que trabaja sobre React, pero como no había soporte para la versión de React sobre la cual desarrollamos el software (React 18), tuvimos que cambiar de herramienta a una más accesible.

Finalmente nos cambiamos a la herramienta Robot Framework, la cual nos permitía realizar pruebas de manera fácil sobre el Front-End, y la cual era independiente del entorno de desarrollo.

En segundo lugar, decidimos el alcance de nuestra aplicación web. De aquí nacieron las siguientes funcionalidades:

RF1: Listar todos los médicos.

RF2: Buscar médico por especialidad.

RF3: Mostrar información de Médico.

RF4: Agregar médico a la plataforma (Funcionalidad sólo para administradores).

RF5: Editar información de médico dentro de la plataforma (Funcionalidad sólo para administradores).

RF6: Eliminar médico de la plataforma (Funcionalidad solo para administradores).

RF7: Login de Administrador.

Y a partir de lo anterior se agregaron como funcionalidades los siguientes supuestos:

RF8: El campo *edad* debe estar entre 1-99 años.

RF9: Largo de los campos *nombre/apellido/especialidad* ≤ 50 caracteres

RF10: No se acepta números, ni tildes ni espacios dentro de los campos *nombre/apellido/especialidad*.

RF11: Los campos *ubicacion/credencial* deben estar en formato url.

RF12: El campo *rut* debe ser un rut válido.

RF13: No pueden existir 2 o más médicos con el mismo *rut*.

En tercer lugar configuramos la herramienta de Git-Hub para ocupar el framework de GitFlow. También enlazamos las herramientas de Github y Jira a Slack con el objetivo de centralizar las notificaciones y así tener un mejor control de *qué está haciendo cada integrante del equipo*. En cuarto lugar pasamos a definir la estrategia de pruebas y el conjunto de estas para así, realizar el software pensando en el resultado que cada funcionalidad debía retornar.

Finalmente pasamos al desarrollo de la aplicación utilizando las herramientas anteriormente mencionadas, en conjunto con los frameworks respectivos en donde no tuvimos mayores complicaciones, hasta que tuvimos que llevar a cabo la fase de pruebas. Debido a la complicación que el framework Enzyme nos exponía, nos cambiamos a Robot Framework en donde se realizaron las pruebas de manera rápida y concisa.

4.2. Dependencias entre herramienta y aplicación

En primer lugar, hablando de la configuración del entorno, se debía tener instalado una versión de Node.js para ejecutar el servidor de desarrollo de React, y como también estaba instalado Django Rest Framework, se contaba con una versión de python actualizada dentro del ambiente de trabajo. Bajo estas condiciones se instaló el framework de Robot. A lo anterior se le debe agregar que Robot trabaja con librerías de Selenium. Estas librerías, le permiten al framework controlar y manipular el navegador web de forma programática, lo que facilita la interacción con los elementos de la interfaz de usuario de la aplicación web.

Tanto para la escritura de casos de prueba como para su ejecución, se ocupó una terminal independiente de las que estaban corriendo la aplicación (Backend y Frontend), en donde finalmente Robot ejecutó los tests y generó los logs correspondientes a cada prueba.

5. Pruebas

5.1. Estrategia de pruebas utilizadas

Para definir las pruebas identificamos las siguientes estrategias:

1. Valores límite:
 - 1.1 Cantidad de caracteres en los campos.
 - 1.2 Edad aceptada (1-99) años.
2. Supuestos:
 - 2.1 Rut válido.
 - 2.2 Campo respeta formato pedido.
3. Pruebas funcionales.
4. Pruebas de Integración.

5.2. Definición de Pruebas

5.2.1. Pruebas unitarias/integración

- Abrir información de un médico.
- Realizar Login (Valid login admin).
- Crear médico (forma válida).
- Crear médico (forma no válida).
- Eliminar un médico.
- Editar Médico (forma válida).
- Editar Médico (forma no válida).
- Buscar médico por especialización.

5.2.2. Pruebas de Valores límite/Supuestos

- Rut válido/no válido (Validado por fórmula).
- Rut válido/no valido (Existe/No existe).
- Edad válida/no válida (0,1,100).
- Nombre válido/no válido (Largo).
- Nombre válido/no válido (Con espacios).
- Apellido válido/no válido (Largo).
- Apellido válido/no válido (Con espacios).
- Especialidad válida/no válida (espacios/largo).
- Credencial válida/no válida.
- Ubicación válida/no válida.

5.3. Procedimiento de ejecución de pruebas

5.3.1. Pruebas unitarias del sistema CRUD

P1: Prueba unitaria de creación de registro:

- 1.1 Se crea un registro ficticio y se llama al método de creación correspondiente.
- 1.2 Se verifica que el registro se haya guardado correctamente en la base de datos.
- 1.3 Se verifica que los datos del registro creado sean los mismos que los proporcionados en la entrada.

P2: Prueba unitaria de lectura de registro:

- 2.1 Se agrega un registro ficticio a la base de datos.
- 2.2 Se llama al método de lectura correspondiente y se proporciona el ID del registro creado anteriormente.
- 2.3 Se verifica que el registro devuelto tenga los mismos valores que el registro creado anteriormente.

P3: Prueba unitaria de actualización de registro:

- 3.1 Se crea un registro ficticio y se guarda en la base de datos.
- 3.2 Se modifica algún campo del registro creado anteriormente y se llama al método de actualización correspondiente.
- 3.3 Se verifica que los cambios se hayan guardado correctamente en la base de datos.
- 3.4 Se verifica que los datos del registro actualizado sean los mismos que los proporcionados en la entrada.

P4: Prueba unitaria de eliminación de registro:

- 4.1 Se crea un registro ficticio y se guarda en la base de datos.
- 4.2 Se llama al método de eliminación correspondiente y se proporciona el ID del registro creado anteriormente.
- 4.3 Se verifica que el registro se haya eliminado correctamente de la base de datos.
- 4.4 Se verifica que el registro eliminado ya no esté presente en la base de datos al intentar leerlo.

5.3.2. Pruebas unitarias de Valores Límite/ Supuestos

P5: Validar Rut (Según fórmula)

- 5.1 Se ingresa un registro ficticio de un médico con un rut inválido (Según fórmula).
- 5.2 Se verifica que no se haya realizado el post del registro al Back-End.
- 5.3 Se ingresa un registro ficticio de un médico con un rut válido (Según fórmula).
- 5.4 Se verifica que se haya realizado el post del registro al Back-End.
- 5.5 Se verifica que los datos del registro creado sean los mismos que los proporcionados en la entrada.

P6: Validar Rut (Existe o no Dentro de la Base de Datos)

- 6.1 Se ingresa un registro ficticio de un médico con un rut que ya exista dentro de la Base de Datos.
- 6.2 Se verifica que el sistema entregue feedback de que el rut proporcionado está actualmente en uso.
- 6.3 Se ingresa un registro ficticio de un médico con un rut válido (Según fórmula), y que no exista en la Base de Datos.
- 6.4 Se verifica que se haya realizado el post del registro al Back-End.
- 6.5 Se verifica que los datos del registro creado sean los mismos que los proporcionados en la entrada.

P7: Validar Edad

- 7.1 Se ingresa un registro ficticio de un médico con una edad igual a 0.
- 7.2 Se verifica que el sistema entregue feedback de que la edad entregada no es correcta.
- 7.3 Se ingresa un registro ficticio de un médico con una edad igual a 100.
- 7.4 Se verifica que el sistema entregue feedback de que la edad entregada no es correcta.
- 7.5 Se ingresa un registro ficticio de un médico con una edad igual a 1.
- 7.6 Se verifica que se haya realizado el post del registro al Back-End.
- 7.7 Se verifica que los datos del registro creado sean los mismos que los proporcionados en la entrada.

P8: Validar nombre/apellido/especialidad

- 8.1 Se ingresa un registro ficticio de un médico con un nombre con largo mayor a 50 caracteres.
- 8.2 Se verifica que el sistema no permita ingresar más de 50 caracteres al campo.
- 8.3 Se ingresa un registro ficticio de un médico con un nombre con largo menor a 50 caracteres con un espacio al inicio.
- 8.4 Se verifica que el sistema entregue feedback de que el nombre entregado no es correcto.
- 8.5 Se ingresa un registro ficticio de un médico con un nombre con largo menor a 50 caracteres que contenga números.
- 8.6 Se verifica que el sistema entregue feedback de que el nombre entregado no es correcto.
- 8.7 Se ingresa un registro ficticio de un médico con un nombre con largo menor a 50 caracteres que contenga un tilde.
- 8.8 Se verifica que el sistema entregue feedback de que el nombre entregado no es correcto.
- 8.9 Se ingresa un registro ficticio de un médico con un nombre con largo menor a 50 caracteres que no contenga tildes ni espacios al inicio.
- 8.10 Se verifica que se haya realizado el post del registro al Back-End.
- 8.11 Se verifica que los datos del registro creado sean los mismos que los proporcionados en la entrada.

P9: Validar Ubicacion/Credencial

- 9.1 Se ingresa un registro ficticio de un médico con ubicacion inválida (No formato url)
- 9.2 Se verifica que el sistema entregue feedback de que la ubicacion entregado no es correcto.
- 9.3 Se ingresa un registro ficticio de un médico con ubicacion válida (Formato url).
- 9.4 Se verifica que se haya realizado el post del registro al Back-End.
- 9.5 Se verifica que los datos del registro creado sean los mismos que los proporcionados en la entrada.

5.4. Reporte de resultados

ID Caso	Entrada	Resultado Esperado	Resultado Obtenido	Fallo o Éxito	Comentarios Adicionales
P1.1	{ "rut": "198762156", "nombre": "Josefa", "apellido_P": "Castro", "apellido_M": "Pinoza", "edad": 24, "especialidad": "Cirujano", "ubicacion": "https://goo.gl/maps/EJocG2LnmFPJNspj7", "credencial": "https://www.soundczech.cz/temp/lorem-ipsun.pdf" }	Registro Creado Coincide con los datos de Entrada	Registro Creado Coincide con los datos de Entrada	Éxito	Usuario debe encontrarse en el formulario de Agregar médico Es necesario que el usuario se encuentre logueado
P1.2	{ "rut": "123456789", "nombre": "Josefa", "apellido_P": "Castro2", "apellido_M": "Pinoza", "edad": 0, "especialidad": "Cirujano", "ubicacion": "https://goo.gl/maps/EJocG2LnmFPJNspj7", "credencial": "https://www.soundczech.cz/temp/lorem-ipsun.pdf" }	Registro no Creado Error en la entrada	Registro no Creado Error en la entrada	Éxito	Usuario debe encontrarse en el formulario de Agregar médico Es necesario que el usuario se encuentre logueado
P2.1	Buscar el registro del Caso P1.1	Registro Encontrado	Registro Encontrado	Éxito	Usuario debe encontrarse en la tabla de médicos Se puede leer el registro creado en P1.1
P2.2	Buscar un Registro que no Exista	Registro No Encontrado	Registro No Encontrado	Éxito	
P3.1	{ "rut": "999999999", "nombre": "Josefa", "apellido_P": "Castillo", "apellido_M": "Pinoza", "edad": 26, "especialidad": "Cirujano", "ubicacion": "https://goo.gl/maps/EJocG2LnmFPJNspj7", "credencial": "https://www.soundczech.cz/temp/lorem-ipsun.pdf" }	Registro Editado Coincide con los datos de Entrada	Registro Editado Coincide con los datos de Entrada	Éxito	Usuario debe encontrarse en la vista de editar médico Se puede modificar el registro del caso P1.1 El nuevo rut, puede ser cualquier rut válido Es necesario que el usuario se encuentre logueado
P3.2	{ "rut": "123456789", "nombre": "Josefa", "apellido_P": "Castillo", "apellido_M": "Espinosa", "edad": 26, "especialidad": "Cirujano", "ubicacion": "https://goo.gl/maps/EJocG2LnmFPJNspj7", "credencial": "https://www.soundczech.cz/temp/lorem-ipsun.pdf" }	Registro no Editado Error en la Entrada	Registro no Editado Error en la Entrada	Éxito	Usuario debe encontrarse en la vista de editar médico Es necesario que el usuario se encuentre logueado
P4	Se elimina el Registro del caso P3.1	Registro Eliminado	Registro Eliminado	Éxito	Se puede usar el registro editado en P3.1 Es necesario que el usuario se encuentre logueado
P5.1	{ "rut": "198762157", "nombre": "Josefa", "apellido_P": "Castro", "apellido_M": "Pinoza", "edad": 24, "especialidad": "Cirujano", "ubicacion": "https://goo.gl/maps/EJocG2LnmFPJNspj7", "credencial": "https://www.soundczech.cz/temp/lorem-ipsun.pdf" }	Rut Inválido Registro no Creado	Rut Inválido Registro no Creado	Éxito	Es necesario que el usuario se encuentre logueado El usuario debe encontrarse en el formulario de Agregar Médico
P5.2	{ "rut": "198762156", "nombre": "Josefa", "apellido_P": "Castro", "apellido_M": "Pinoza", "edad": 24, "especialidad": "Cirujano", "ubicacion": "https://goo.gl/maps/EJocG2LnmFPJNspj7", "credencial": "https://www.soundczech.cz/temp/lorem-ipsun.pdf" }	Registro Creado Coincide con los datos de Entrada	Registro Creado Coincide con los datos de Entrada	Éxito	Es necesario que el usuario se encuentre logueado Es necesario que el rut no esté en uso El usuario debe encontrarse en el formulario de Agregar Médico
P6.1	{ "rut": "198762156", "nombre": "Josefa", "apellido_P": "Castro", "apellido_M": "Pinoza", "edad": 24, "especialidad": "Cirujano", "ubicacion": "https://goo.gl/maps/EJocG2LnmFPJNspj7", "credencial": "https://www.soundczech.cz/temp/lorem-ipsun.pdf" }	Rut Actualmente en Uso Registro no Creado	Rut Actualmente en Uso Registro no Creado	Éxito	Es necesario que el usuario se encuentre logueado El usuario debe encontrarse en el formulario de Agregar Médico
P6.2	{ "rut": "999999999", "nombre": "Josefa", "apellido_P": "Castro", "apellido_M": "Pinoza", "edad": 24, "especialidad": "Cirujano", "ubicacion": "https://goo.gl/maps/EJocG2LnmFPJNspj7", "credencial": "https://www.soundczech.cz/temp/lorem-ipsun.pdf" }	Registro Creado Coincide con los datos de Entrada	Registro Creado Coincide con los datos de Entrada	Éxito	Es necesario que el usuario se encuentre logueado El usuario debe encontrarse en el formulario de Agregar Médico

ID Caso	Entrada	Resultado Esperado	Resultado Obtenido	Fallo o Éxito	Comentarios Adicionales
P7.1	{ "rut": "198762156", "nombre": "Josefa", "apellido_P": "Castro", "apellido_M": "Pinoza", "edad": 0, "especialidad": "Cirujano", "ubicacion": "https://goo.gl/maps/EJocG2LnmFPJNspj7", "credencial": "https://www.soundcezech.cz/temp/lorem-ipsum.pdf" }	Registro no Creado Error en el campo Edad	Registro no Creado Error en el campo Edad	Éxito	Es necesario que el usuario se encuentre logueado El usuario debe encontrarse en el formulario de Agregar Médico
P7.2	{ "rut": "198762156", "nombre": "Josefa", "apellido_P": "Castro", "apellido_M": "Pinoza", "edad": 100, "especialidad": "Cirujano", "ubicacion": "https://goo.gl/maps/EJocG2LnmFPJNspj7", "credencial": "https://www.soundcezech.cz/temp/lorem-ipsum.pdf" }	Registro no Creado Error en el campo Edad	Registro no Creado Error en el campo Edad	Éxito	Es necesario que el usuario se encuentre logueado El usuario debe encontrarse en el formulario de Agregar Médico
P7.3	{ "rut": "198762156", "nombre": "Josefa", "apellido_P": "Castro", "apellido_M": "Pinoza", "edad": 1, "especialidad": "Cirujano", "ubicacion": "https://goo.gl/maps/EJocG2LnmFPJNspj7", "credencial": "https://www.soundcezech.cz/temp/lorem-ipsum.pdf" }	Registro Creado Coincide con los datos de Entrada	Registro Creado Coincide con los datos de Entrada	Éxito	Es necesario que el usuario se encuentre logueado El usuario debe encontrarse en el formulario de Agregar Médico
P8.1	{ "rut": "222222222", "nombre": "[más de 50 caracteres]", "apellido_P": "Castro", "apellido_M": "Pinoza", "edad": 25, "especialidad": "Cirujano", "ubicacion": "https://goo.gl/maps/EJocG2LnmFPJNspj7", "credencial": "https://www.soundcezech.cz/temp/lorem-ipsum.pdf" }	El campo no permite ingresar mas de 50 caracteres	El campo no permite ingresar mas de 50 caracteres	Éxito	Es necesario que el usuario se encuentre logueado El usuario debe encontrarse en el formulario de Agregar Médico
P8.2	{ "rut": "222222222", "nombre": "[menos de 50 caracteres con espacio al inicio]", "apellido_P": "Castro", "apellido_M": "Pinoza", "edad": 25, "especialidad": "Cirujano", "ubicacion": "https://goo.gl/maps/EJocG2LnmFPJNspj7", "credencial": "https://www.soundcezech.cz/temp/lorem-ipsum.pdf" }	El campo no permite tildes en la entrada	El campo no permite tildes en la entrada	Éxito	Es necesario que el usuario se encuentre logueado El usuario debe encontrarse en el formulario de Agregar Médico
P8.3	{ "rut": "222222222", "nombre": "[menos de 50 caracteres que contenga un tilde]", "apellido_P": "Castro", "apellido_M": "Pinoza", "edad": 25, "especialidad": "Cirujano", "ubicacion": "https://goo.gl/maps/EJocG2LnmFPJNspj7", "credencial": "https://www.soundcezech.cz/temp/lorem-ipsum.pdf" }	El campo no permite tildes en la entrada	El campo no permite tildes en la entrada	Éxito	Es necesario que el usuario se encuentre logueado El usuario debe encontrarse en el formulario de Agregar Médico
P8.4	{ "rut": "222222222", "nombre": "Juan", "apellido_P": "Castro", "apellido_M": "Pinoza", "edad": 25, "especialidad": "Cirujano", "ubicacion": "https://goo.gl/maps/EJocG2LnmFPJNspj7", "credencial": "https://www.soundcezech.cz/temp/lorem-ipsum.pdf" }	Registro Creado Coincide con los datos de Entrada	Registro Creado Coincide con los datos de Entrada	Éxito	Es necesario que el usuario se encuentre logueado El usuario debe encontrarse en el formulario de Agregar Médico
P9.1	{ "rut": "333333333", "nombre": "Juan", "apellido_P": "Castro", "apellido_M": "Pinoza", "edad": 25, "especialidad": "Cirujano", "ubicacion": "[Entrada no esta formato url]", "credencial": "https://www.soundcezech.cz/temp/lorem-ipsum.pdf" }	El campo solo permite url en la entrada	El campo solo permite url en la entrada	Éxito	Es necesario que el usuario se encuentre logueado El usuario debe encontrarse en el formulario de Agregar Médico
P9.2	{ "rut": "333333333", "nombre": "Juan", "apellido_P": "Castro", "apellido_M": "Pinoza", "edad": 25, "especialidad": "Cirujano", "ubicacion": "https://goo.gl/maps/EJocG2LnmFPJNspj7", "credencial": "https://www.soundcezech.cz/temp/lorem-ipsum.pdf" }	Registro Creado Coincide con los datos de Entrada	Registro Creado Coincide con los datos de Entrada	Éxito	Es necesario que el usuario se encuentre logueado El usuario debe encontrarse en el formulario de Agregar Médico

6. Problemas encontrados y soluciones

El mayor problema, fue la elección de la herramienta de testing. Dado que en un comienzo escogimos Enzyme, y como su compatibilidad llegaba hasta React 16, no era compatible con nuestro desarrollo utilizando la versión 18. Para solucionar este inconveniente, se decidió cambiar de software de testing a Robot Framework, el cual, al realizar Web Scrapping, no depende del entorno de desarrollo y permite testear cualquier aplicación web.

7. Conclusión

Dadas las limitaciones de tiempo, el proyecto y la página web desarrollada se enfocan en la visualización correcta de los médicos, sus características como especialidad y credenciales de estos para así entregar una plataforma web acorde a las necesidades de las personas que buscaban obtener una hora de consulta médica con un especialista, además se agregaron la adición y modificación de los médicos a fin de poder añadirlos según fuera necesario para la clínica.

En términos de las tecnologías utilizadas, solo un miembro posee altos conocimientos de estas, pero para el resto de los integrantes conocer Django fue una experiencia interesante dado el alto grado de funcionamiento de Django y el uso de las APIs lo cual facilitaba enormemente la integración de otros lenguajes de programación como también la independización de la funcionalidad del código separando Front-End y Back-End. En cuanto a React, su uso fue llamativo y la capacidad de crear estructuras visuales de manera sencilla ayuda enormemente a desarrolladores poco experimentados en Front-End.

Por último en las pruebas, Enzyme nos causó un gran problema dada su incompatibilidad de uso en proyectos basados en React version 18, no obstante se pudo continuar con el trabajo sustituyendo la herramienta por Robot Framework, la cuál todos los integrantes concluimos que posee un gran uso para las pruebas en base a Front-End siendo especialmente útil en situaciones donde se deba realizar una demo de pruebas para un cliente dado que muestra todo el proceso de las pruebas a realizar y las cuales son sencillas de crear en cuanto a código, aunque su instalación puede ser algo más tediosa de lo que se esperaba.